Project 2: Extending Apache Roller with Advanced Features and Design Patterns

CS6.401 – Software Engineering

TEAM 33
Members
- Aryan Mishra, 2024121001
- Aditya Nair, 2023111029
- Varun Patil, 2025204013
- Hardik Chadha, 2023111031
- Vijay Aravynthan, 2023111007

Repository: https://github.com/serc-courses/project-1-team-33

Branch: project2_33

---

1. Introduction

In this project, we extended the functionality of the Apache Roller blogging platform by implementing several new features aimed at improving user engagement, administrative monitoring, and content processing. The goal of this phase was to enhance the existing system while maintaining good software engineering practices such as modularity, extensibility, and maintainability.

The features implemented in this project include a **User Highlights system for starring blogs and identifying trending content**, a **content transformation pipeline for preprocessing blog posts**, a **bug reporting system with administrative management**

and notifications, an **admin analytics dashboard**, **webpage translation with caching**, and a **community pulse dashboard that summarizes comment discussions**.

While implementing these features, emphasis was placed on applying appropriate **software design patterns** to ensure that the system remains flexible and easy to extend in the future. Each feature was designed as a modular component integrated with the existing Apache Roller architecture.

2. System Overview

Apache Roller is a Java-based blogging platform that allows users to create weblogs, publish blog posts, and interact with readers through comments and discussions. The system supports multiple users and blogs, making it suitable for collaborative blogging environments.

The core system includes modules responsible for **user management, weblog entry management, comment handling, and template-based blog rendering**. These components provide the foundation upon which additional features can be built.

In this project, we extended the existing system by adding new modules and services that integrate with the current architecture. The new components interact with the existing data models and service layers of Apache Roller while keeping the implementation modular and loosely coupled.

The implemented features focus on improving different aspects of the platform:

- increasing **user engagement** through starring and trending blogs
- enabling **automated content processing** through feed transformation pipelines
- allowing **users to report issues** and enabling administrators to manage them
- providing **administrative insights** through a dashboard with site metrics
- enabling **multilingual accessibility** through translation features
- summarizing **community discussions** through comment analysis tools

These additions enhance the usability and functionality of the platform while maintaining compatibility with the existing Roller system architecture.

3. Feature Implementations

Task 1: User Highlights (Stars and Trending)

Task 1A — Stars Feature

---

1. Feature Description

The Stars feature allows authenticated users to "star" (favourite) blog pages (weblogs) and individual blog posts (entries). Users can view all their starred content from dedicated pages accessible via the main menu sidebar. Starred weblogs are sorted by most recent blog post date/time, and starred blog posts support pagination (30 items per page). The feature adds a user engagement layer to Apache Roller, enabling users to bookmark and quickly access their favourite content.

---

2. Implementation

Backend

The backend follows Roller's existing manager-based architecture with JPA persistence.

Domain Model: Two new JPA entities were created to represent the star relationships:

`UserWeblogStar` — maps a user to a weblog with a starredTime timestamp. Fields: id (String), user (User), weblog (Weblog), starredTime (Timestamp). The entity is mapped to the user_weblog_star table with a unique constraint on (userid, weblogid).

`UserEntryStar` — maps a user to a blog entry with a starredTime timestamp. Fields: id (String), user (User), entry (WeblogEntry), starredTime (Timestamp). Mapped to the user_entry_star table with a unique constraint on (userid, entryid).

Both entities use UUIDs as primary keys and have proper foreign key constraints referencing roller_user, weblog, and weblogentry tables.

Manager Interface and Implementation:

The `StarManager` interface defines 15 methods organized into five groups:

Weblog star operations:
`saveWeblogStar()`, `removeWeblogStar()`, `getWeblogStar()`, `getWeblogStarByUserAndWeblog()`, `getStarredWeblogs(user, offset, length)`

Entry star operations:
`saveEntryStar()`, `removeEntryStar()`, `getEntryStar()`, `getEntryStarByUserAndEntry()`, `getStarredEntries(user, offset, length)`

Trending aggregates:
`getTrendingWeblogs(limit)`, `getTrendingEntries(limit)`

Counts:
`getWeblogStarCount(weblog)`, `getEntryStarCount(entry)`

Lifecycle:
`release()`

The JPA implementation `JPAStarManagerImpl` uses named queries defined in ORM XML files:

`UserWeblogStar.getStarredWeblogsByUser` — SELECT s FROM UserWeblogStar s WHERE s.user = ?1 ORDER BY s.weblog.lastModified DESC

`UserEntryStar.getStarredEntriesByUser` — SELECT s FROM UserEntryStar s WHERE s.user = ?1 ORDER BY s.starredTime DESC

Struts2 Actions:

`StarWeblogAction` — handles `star()` and `unstar()` for weblogs. Prevents users from starring their own blog. Redirects back to the blog page after the action.

`StarEntryAction` — handles `star()` and `unstar()` for entries. Redirects to the entry's permalink after the action.

`StarredWeblogsAction` — loads the authenticated user's starred weblogs, sorted by weblog.lastModified (most recent post first).

`StarredEntriesAction` — loads the user's starred entries with pagination support (PAGE_SIZE = 30). Fetches PAGE_SIZE + 1 items to detect whether a "Next" page exists.

`StarredEntriesPager` — implements next/previous page navigation with proper page parameter handling.

Velocity Template Integration:

The `UtilitiesModel` class provides two methods for Velocity templates:

`isWeblogStarred(weblogHandle)` — checks if the current user has starred a weblog

`isEntryStarred(entryId)` — checks if the current user has starred an entry

Both methods safely return false for anonymous (unauthenticated) users.

Frontend

Weblog Star Button (`sidebar.vm`): The basic theme's sidebar template displays a star/unstar button for authenticated users when viewing a weblog. The button is hidden for the blog creator (cannot self-star). The star action submits a form to the `starWeblog` Struts action.

Entry Star Button (`permalink.vm`): The permalink template for individual blog posts displays a star/unstar button for authenticated users. Similar to the weblog button, it submits to the `starEntry` Struts action.

Starred Weblogs Page (`StarredWeblogs.jsp`): Displays a table with columns: Weblog Name, Last Post Date (formatted as yyyy-MM-dd HH:mm), and an Unstar button. If no starred weblogs exist, a friendly message is shown.

Starred Entries Page (`StarredEntries.jsp`): Displays starred blog posts with title, blog name, and starred date. Includes Previous/Next pagination controls at the bottom.

Main Menu Sidebar (`MainMenuSidebar.jsp`): Adds links to "Starred Weblogs" and "Starred Entries" pages in the user's main navigation sidebar.

[Screenshots to be added: weblog star button on sidebar, entry star button on permalink page, starred weblogs list, starred entries list with pagination]

---

3. Assumptions / Constraints

Only authenticated users can star content; anonymous visitors see no star buttons

Users cannot star their own weblogs (self-starring is prevented in `StarWeblogAction`)

Star operations are idempotent — starring an already-starred item is a no-op

The starred weblogs page sorts by weblog.lastModified, which reflects the most recent blog post date

Starred entries page uses a fixed page size of 30 entries per page

Star data is stored in dedicated database tables with proper foreign key constraints and unique indexes to prevent duplicate stars

---

4. Files Created and Modified

New Files Created

Domain and Persistence:
`UserWeblogStar.java`
`UserEntryStar.java`
`UserWeblogStar.orm.xml`
`UserEntryStar.orm.xml`

Business Logic:
`StarManager.java`
`JPAStarManagerImpl.java`

UI Actions:
`StarWeblogAction.java`
`StarEntryAction.java`
`StarredWeblogsAction.java`
`StarredEntriesAction.java`
`StarredEntriesPager.java`

JSP Views:
`StarredWeblogs.jsp`
`StarredEntries.jsp`

Tests:
`StarManagerTest.java`
`UserWeblogStarTest.java`
`UserEntryStarTest.java`

Files Modified

`struts.xml` — added action mappings for starredWeblogs, starredEntries, starWeblog, starEntry

`tiles.xml` — added tile definitions for .StarredWeblogs, .StarredEntries

`sidebar.vm` — added star/unstar button for weblogs

`permalink.vm` — added star/unstar button for entries

`MainMenuSidebar.jsp` — added links to starred pages

`UtilitiesModel.java` — added `isWeblogStarred()` and `isEntryStarred()` methods

`createdb.vm` — added user_weblog_star and user_entry_star tables, indexes, and foreign keys

`ApplicationResources.properties` — added i18n messages for star UI

`Weblogger.java` — added `getStarManager()` to the main interface

`WebloggerImpl.java` — added `StarManager` field

`JPAWebloggerImpl.java` — registered `JPAStarManagerImpl`

`JPAWebloggerModule.java` — Guice binding for `StarManager`

---

5. Design Patterns Used

Repository Pattern: The `StarManager` interface and `JPAStarManagerImpl` implementation follow the Repository pattern, consistent with Roller's existing manager architecture. All persistence logic for star entities is encapsulated behind the `StarManager` interface, isolating the rest of the application from JPA-specific details. This ensures that database operations are centralized and consistent.

Observer Pattern (Implicit): The Struts2 action → manager → JPA flow follows an implicit observer-like pattern where the UI layer notifies the business layer of state changes (star/unstar events), and the persistence layer reacts accordingly.

---

6. Rationale Behind Pattern Selection

The Repository pattern was chosen because it aligns with Roller's existing architecture — every data entity in Roller (users, weblogs, entries, comments) is managed through a dedicated Manager interface with a JPA implementation. Following this established convention ensures consistency, makes the code immediately familiar to anyone who has worked with Roller's codebase, and leverages the existing Guice dependency injection and transaction management infrastructure.

---

7. User Flow

Starring a Weblog:

User navigates to a public weblog page
If authenticated and not the blog owner, a "Star" button appears in the sidebar
User clicks "Star" → form submits to `starWeblog!star` action
`StarWeblogAction.star()` creates a `UserWeblogStar` record via `StarManager`
User is redirected back to the same weblog page; button now shows "Unstar"

Viewing Starred Content:

User clicks "Starred Weblogs" or "Starred Entries" in the main menu sidebar
The respective action loads the user's starred items from the database
Starred weblogs are displayed sorted by most recent post date
Starred entries are displayed with pagination (30 per page)
User can click "Unstar" to remove an item from their favourites

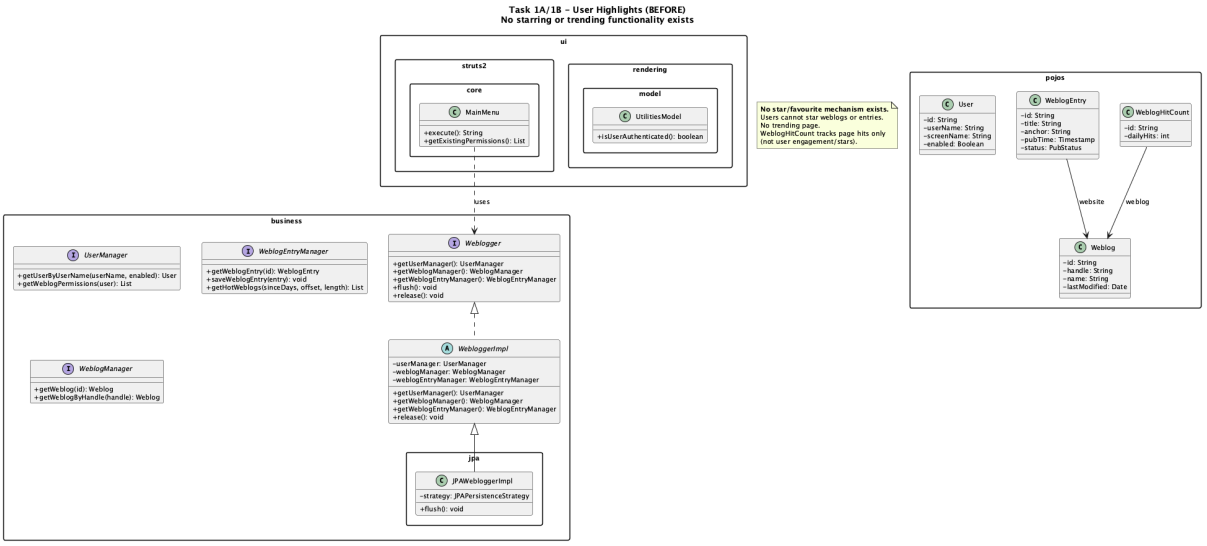
---

8. Before and After UML Class Diagram

Before (no star/favourite mechanism exists):

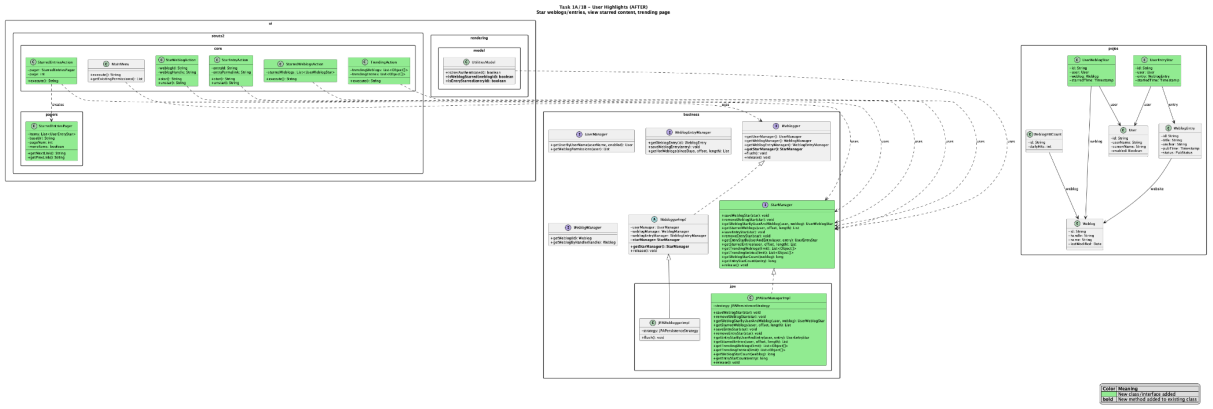
The existing system has User, Weblog, WeblogEntry, and WeblogHitCount entities. WeblogHitCount tracks page hits only — there is no mechanism for users to star or favourite

content, and no trending page.



After (full star and trending system):

New entities `UserWeblogStar` and `UserEntryStar` link users to content. The `StarManager` interface and `JPAStarManagerImpl` provide persistence. Five new Struts2 actions handle starring, listing starred content, and trending pages. The `UtilitiesModel` exposes star-checking methods to Velocity templates.



Task 1B — Trending Blogs

1. Feature Description

The Trending Blogs feature displays the top 5 most-starred weblogs and blog posts across the entire platform. It provides a public-facing page that highlights the most popular content based on star counts, enabling content discovery and community engagement. The implementation uses efficient aggregate SQL queries (GROUP BY + COUNT) to avoid iterating over individual articles.

---

2. Implementation

Backend

Struts2 Action:

`TrendingAction` defines `TRENDING_LIMIT = 5` and calls:

`getTrendingWeblogs(5)` — returns top 5 weblogs by star count

`getTrendingEntries(5)` — returns top 5 entries by star count

Both methods return `List<Object[]>` where [0] is the entity (Weblog or WeblogEntry) and [1] is the star count (Long).

Efficient Aggregate Queries (Named Queries in ORM XML):

Trending weblogs query:

SELECT s.weblog.id, COUNT(s) FROM UserWeblogStar s GROUP BY s.weblog.id ORDER BY COUNT(s) DESC

Trending entries query:

SELECT s.entry.id, COUNT(s) FROM UserEntryStar s GROUP BY s.entry.id ORDER BY COUNT(s) DESC

These queries use GROUP BY + COUNT at the database level, which is efficient because:

Only one SQL query is executed per trending list (not N+1)

The database engine handles the sorting and limiting

Only the top N entity IDs are resolved to full objects (via strategy.load())

Frontend

Trending Page (`Trending.jsp`): Displays two sections:

"Top 5 Trending Weblogs" — table with rank (#), weblog name (linked), and star count in a badge

"Top 5 Trending Blog Posts" — table with rank, entry title (linked to permalink), blog name, and star count badge

The page is accessible from the main menu sidebar.

[Screenshots to be added: trending page showing top 5 weblogs and entries with star counts]

---

3. Assumptions / Constraints

Trending is based solely on star count — no time decay, no view count component

If no weblogs or entries have been starred, the respective sections show an empty message

The page is accessible to all users (including anonymous visitors)

Only the top 5 items are shown per category

Efficient aggregate queries ensure the feature scales to large datasets without performance degradation

---

4. Files Created and Modified

New Files Created:

`TrendingAction.java`
`Trending.jsp`

Files Modified:

`struts.xml` — added trending action mapping

`tiles.xml` — added .Trending tile definition

`MainMenuSidebar.jsp` — added "Trending" link

`UserWeblogStar.orm.xml` — added `getTrendingWeblogs` named query

`UserEntryStar.orm.xml` — added `getTrendingEntries` named query

`ApplicationResources.properties` — added i18n messages

---

5. Design Patterns Used

Repository Pattern: The trending queries are encapsulated within `StarManager` / `JPAStarManagerImpl`, keeping the database query logic isolated from the UI layer.

---

6. Rationale Behind Pattern Selection

The trending feature is a natural extension of the `StarManager` repository. By keeping the aggregate queries inside the same manager, we avoid creating a separate service layer for what is essentially a different read projection of the same star data. The Struts2 action remains thin (just calls the manager and forwards to JSP), consistent with Roller's MVC architecture.

---

7. User Flow

User clicks "Trending" in the main menu sidebar

`TrendingAction.execute()` calls `getTrendingWeblogs(5)` and `getTrendingEntries(5)`

Both queries execute efficient GROUP BY + COUNT SQL at the database level

Only the top 5 entity IDs are resolved to full Weblog/WeblogEntry objects

Results are displayed in `Trending.jsp` page with rank, name, and star count

---

8. Before and After UML Class Diagram

Same diagrams as Task 1A (the trending feature is part of the same star system).

Task 2: Transforming feeds

---

1. Feature Description

The Transforming Feeds feature adds an admin-side entry processing pipeline that automatically processes blog entries at save time. The pipeline implements the Chain of Responsibility design pattern, where a sequence of independent processing steps transform a `WeblogEntry` in-place before it is persisted to the database.

Five processing steps are implemented:

Profanity Filter — replaces profane words with asterisks

Content Summarizer — generates AI-powered or extractive summaries

Sentiment Analysis — classifies entry sentiment as Positive/Neutral/Negative

Reading Time Estimator — calculates estimated reading time

Auto Tag Generator — extracts keywords as auto-generated tags

Each step can be independently enabled/disabled via roller.properties configuration. Steps can be added or removed from the pipeline without affecting others.

---

2. Implementation

Backend

Core Pipeline Infrastructure:

The `EntryProcessingStep` interface defines the contract for all processing steps:

```
public interface EntryProcessingStep {
   String getName();
   String getDescription();
   void process(WeblogEntry entry);
}
```

The `EntryProcessingPipeline` class orchestrates the chain:

Maintains an ordered `ArrayList<EntryProcessingStep>`

`addStep(step)` — appends a step to the pipeline

`removeStep(stepName)` — removes a step by name (returns boolean)

`getSteps()` — returns unmodifiable view of current steps

`execute(entry)` — runs all steps sequentially; exceptions in one step are caught and logged without breaking the pipeline

---

The `EntryProcessingPipelineFactory` creates a pipeline from admin configuration:

Reads pipeline.step.<name>.enabled properties from roller.properties

Supports configurable parameters per step (e.g., wordsPerMinute, maxTags)

Defines the execution order: ProfanityFilter → ContentSummarizer → SentimentAnalysis → ReadingTimeEstimator → AutoTagGenerator

---

Step 1: `ProfanityFilterStep`

Maintains a static list of profane words with precompiled regex patterns

Performs case-insensitive whole-word matching

Replaces profane words with asterisks (e.g., "damn" → "****")

Modifies entry title, text, and summary fields in-place

Step 2: `ContentSummarizerStep`

Primary mode: calls the Gemini 2.0 Flash API (configured via pulse.llm.apiKey and pulse.llm.apiUrl) to generate a concise summary

Fallback mode: if API is unavailable, extracts the first 2-3 sentences as a summary

Skips entries with ≤2 sentences (too short to summarize)

Strips HTML before analysis; stores summary in entry.summary without modifying original text

Respects a 3000-character limit for API token usage

Step 3: `SentimentAnalysisStep`

Deterministic keyword-based sentiment analysis (no API calls)

Contains 41 positive keywords (great, excellent, amazing, love, wonderful, etc.)

Contains 39 negative keywords (terrible, awful, horrible, hate, disappointing, etc.)

Computes score = positiveCount - negativeCount

Classification: score > 1 = "Positive", score < -1 = "Negative", otherwise "Neutral"

Stores sentiment label in entry.searchDescription metadata field (e.g., "Sentiment: Positive")

Step 4: `ReadingTimeEstimatorStep`

Calculates reading time based on word count divided by WPM (default: 238, configurable)

Strips HTML before counting words

Minimum result is 1 minute

Stores reading time in entry.searchDescription metadata field (e.g., "1 min read")

Step 5: `AutoTagGeneratorStep`

Extracts top N most frequent meaningful words from the entry text (default: 5, configurable)

Strips HTML before analysis

Filters out 70+ English stop words (the, and, for, etc.) and words shorter than 4 characters

Uses term frequency to rank candidate keywords

Appends auto-generated tags as an HTML section with CSS class auto-tags to the entry text

---

Integration with Publishing Flow:

The pipeline is invoked in `EntryEdit.save()` (the Struts2 action for saving blog entries):

EntryProcessingPipeline pipeline = EntryProcessingPipelineFactory.createPipeline();
pipeline.execute(weblogEntry);

This runs after user form data is copied to the `WeblogEntry` object and before the entry is persisted to the database. Both draft saves and published entries pass through the pipeline.

Frontend

The pipeline is entirely backend-driven — there is no dedicated UI. The effects are visible in the published blog posts:

Profane words appear as asterisks

AI-generated or extractive summaries appear in the entry summary field

Sentiment and reading time metadata is stored in searchDescription

Auto-generated tags appear at the bottom of entries

Configuration is managed through roller.properties by the admin.

[Screenshots to be added: a blog post showing auto-generated tags at the bottom, sentiment metadata in the entry summary]

---

3. Assumptions / Constraints

The pipeline runs at save time (not at render time), so changes are persisted to the database

Each step is independent — failure in one step does not prevent others from executing

The profanity filter uses a static word list; it does not detect creative obfuscation

The content summarizer requires a valid Gemini API key for AI-powered summaries; without it, extractive (sentence-based) summarization is used

Sentiment analysis is keyword-frequency based and may not capture sarcasm or nuanced language

Reading time assumes an average reading speed of 238 words per minute (configurable)

Auto-tag generation uses simple term frequency, not TF-IDF or semantic analysis

All steps are enabled by default in roller.properties

---

4. Files Created and Modified

New Files Created:

Pipeline Core:

```
EntryProcessingStep.java
EntryProcessingPipeline.java
EntryProcessingPipelineFactory.java
```

Step Implementations:

```
ProfanityFilterStep.java
ContentSummarizerStep.java
SentimentAnalysisStep.java
ReadingTimeEstimatorStep.java
AutoTagGeneratorStep.java
```

Tests:

```
EntryProcessingPipelineTest.java
ProfanityFilterStepTest.java
ContentSummarizerStepTest.java
SentimentAnalysisStepTest.java
ReadingTimeEstimatorStepTest.java
AutoTagGeneratorStepTest.java
```

Files Modified:

`EntryEdit.java` — added pipeline invocation in `save()` method (lines 214-218)

`roller.properties` — added 7 pipeline configuration properties (5 enabled/disabled flags + 2 parameter configs for wordsPerMinute and maxTags)

---

5 .Design Patterns Used

Chain of Responsibility Pattern: The `EntryProcessingStep` interface defines the contract for each handler in the chain. The `EntryProcessingPipeline` maintains the ordered list of handlers and delegates processing to each one sequentially. Each step is completely independent — it does not know about other steps in the chain, and it can be added or removed without affecting the rest of the pipeline.

Key characteristics of the implementation:

Each step processes the `WeblogEntry` in-place and passes it implicitly to the next step

Steps can be dynamically added (`addStep()`) or removed (`removeStep()`) at runtime

Exception handling per step ensures one failure doesn't break the chain

The factory creates the chain from configuration, making it configurable without code changes

---

Factory Pattern: The `EntryProcessingPipelineFactory` is a static factory that reads configuration properties and assembles the appropriate pipeline. This centralizes pipeline creation logic and keeps step instantiation out of the action layer.

---

6. Rationale Behind Pattern Selection

The Chain of Responsibility pattern was chosen because it directly models the problem: a sequence of independent processing steps that each transform a blog entry. The pattern allows:

Open/Closed Principle: New steps can be added by implementing `EntryProcessingStep` and registering in the factory — no existing code needs to change

Single Responsibility: Each step has one job (filter profanity, summarize, etc.) and knows nothing about other steps

Configurability: Steps can be independently enabled/disabled via properties, and the order is controlled by the factory

Fault isolation: Each step is wrapped in a try-catch, so one failing step doesn't prevent others from running

The Factory pattern complements Chain of Responsibility by separating pipeline construction from usage, and allows the pipeline composition to be driven by admin configuration rather than hardcoded in the action.

---

7. User Flow

Publishing a Blog Entry:

User creates or edits a blog entry in the Roller editor

User clicks "Save Draft" or "Publish"

`EntryEdit.save()` copies form data to the `WeblogEntry` object

`EntryProcessingPipelineFactory.createPipeline()` reads roller.properties and assembles the pipeline with enabled steps

`pipeline.execute(weblogEntry)` runs each step in order:

a. `ProfanityFilterStep` scans and replaces profane words

b. `ContentSummarizerStep` generates summary

c. `SentimentAnalysisStep` classifies sentiment

d. `ReadingTimeEstimatorStep` calculates reading time

e. `AutoTagGeneratorStep` extracts keywords and appends tags

The processed entry is saved to the database

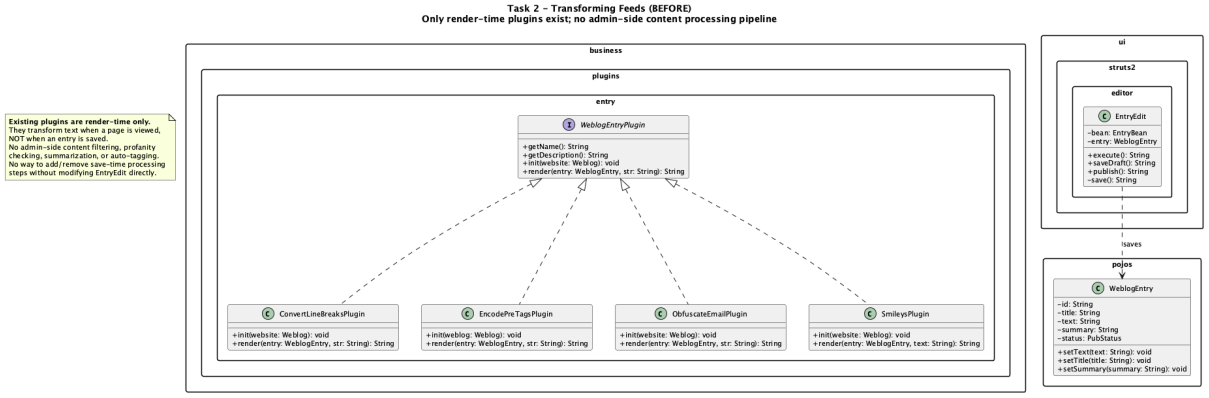
When readers view the blog post, they see cleaned text, auto-tags, and metadata

---

8. Before and After UML Class Diagram

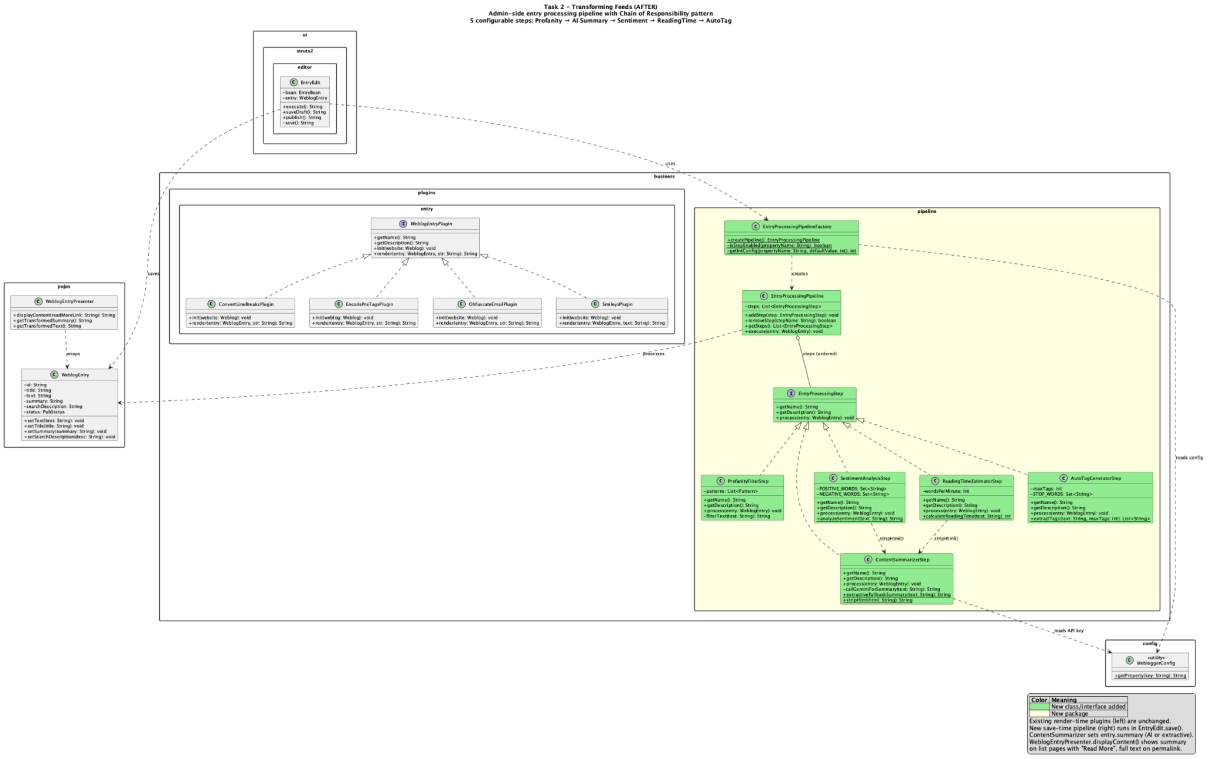
Before (only render-time plugins exist):

The existing system has render-time `WeblogEntryPlugin` implementations (ConvertLineBreaks, EncodePreTags, ObfuscateEmail, Smileys) that transform text when a page is viewed. There is no admin-side content processing at save time, no profanity filtering, no summarization, and no auto-tagging.



After (full Chain of Responsibility pipeline):

The new pipeline package adds the `EntryProcessingStep` interface, `EntryProcessingPipeline` orchestrator, `EntryProcessingPipelineFactory`, and five concrete step implementations. The pipeline is invoked from `EntryEdit.save()` and processes entries at save time, complementing the existing render-time plugins.



Task 3: Bug Reporting system

---

1. Feature Description

The Bug Report module extends Apache Roller with a complete workflow for submitting, tracking, and managing software issue reports within the application.

Regular users can create, view, edit, and delete their own bug reports from the authoring interface. Administrators can review all reports from a centralized dashboard, filter them by status, add administrative notes, change report status, and delete reports when required.

The feature is designed to integrate with existing Roller architecture rather than introducing a separate subsystem. It reuses Struts 2 UI flow, JPA-based manager patterns, configuration systems, and existing security mechanisms.

In addition to CRUD functionality, the module includes an extensible notification pipeline that allows the system to notify administrators and reporters via email, with support for future extensions such as Slack or Microsoft Teams.

---

2. Implementation

Backend

The backend is structured into four layers:

Domain Model
The `BugReport` entity stores report details such as title, description, severity, type, owner, timestamps, status, page URL, reproduction steps, expected/actual behavior, and admin notes.

Data Access Layer
The `BugReportManager` interface and `JPABugReportManagerImpl` handle persistence using Roller's manager-based architecture.

Workflow Layer
The `BugReportWorkflowService` manages validation, command execution, event creation, and notification triggering.

Notification Layer
The `BugReportNotificationService`, `BugReportEventPublisher`,

`BugNotificationChannel`, and `BugNotificationMessageStrategy` handle modular and extensible notification dispatch.

Backend Functionality

- `create()` validates input, stores the report, and notifies admins
- `update()` validates ownership and updates report
- `changeStatus()` validates transitions and notifies both admin and reporter
- `delete()` removes the report and triggers notifications

Admin recipient resolution follows:

`admin users → site.adminemail → mail.username`

Notification delivery is currently email-based using `EmailBugNotificationChannel`, but is abstracted through `BugNotificationChannel` for extensibility.

Status Lifecycle

- OPEN → TRIAGED
- TRIAGED → RESOLVED
- RESOLVED → TRIAGED (reopen)

Frontend

The frontend is implemented using Struts 2 actions and JSP views integrated into Roller UI.

User Features:

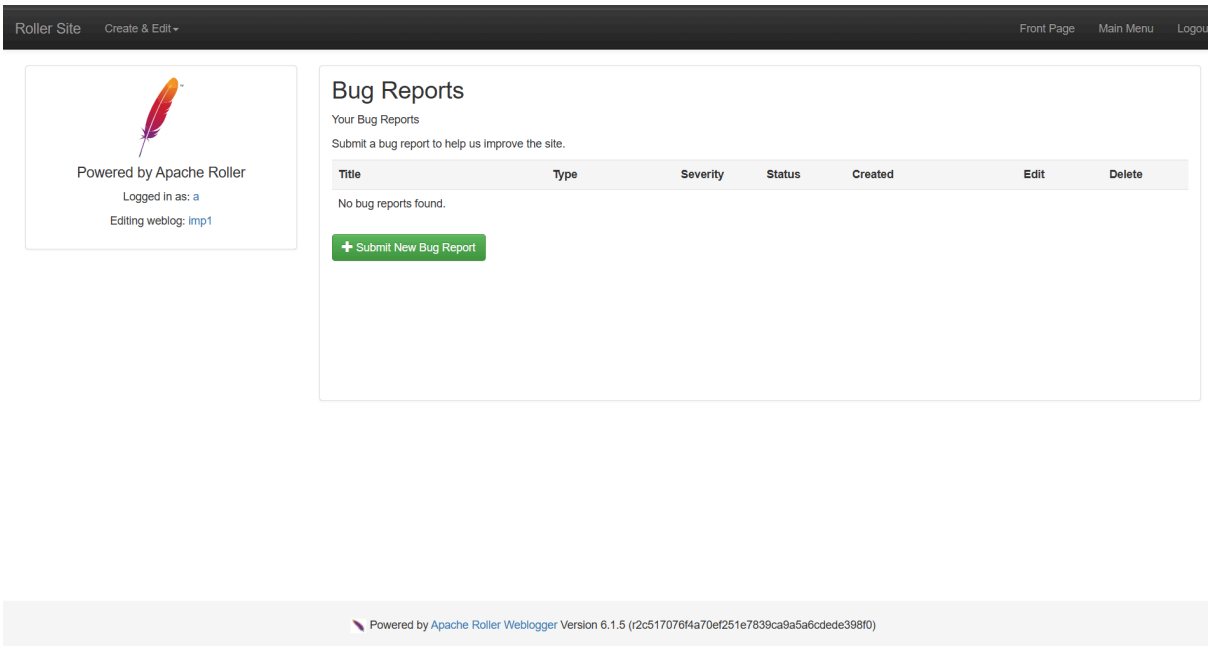
- View bug reports
- Create/edit reports
- Delete owned reports

Admin Features:

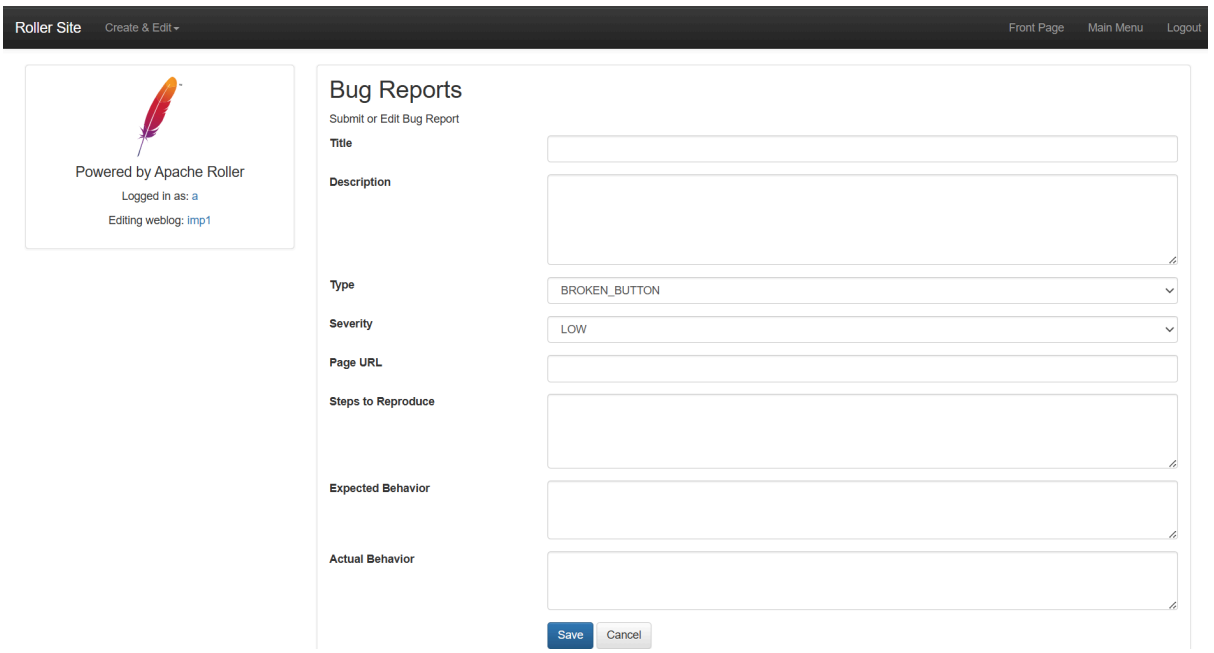
- Central dashboard
- Filter by status
- Update status with notes
- Delete reports

Screenshots

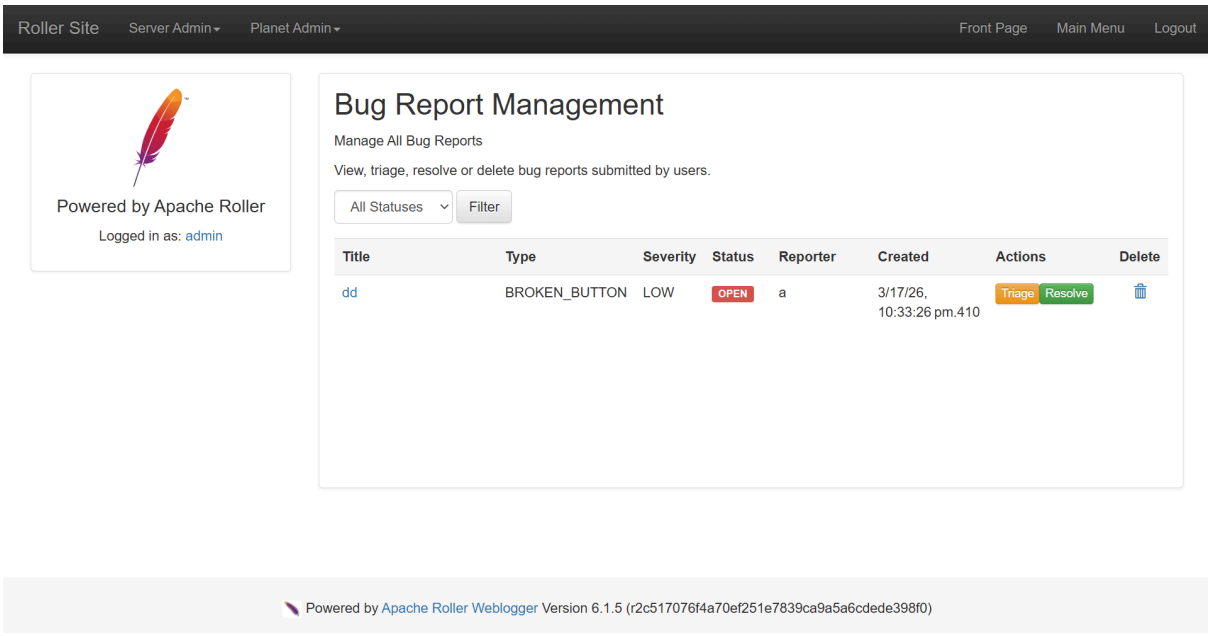
- User bug report list page



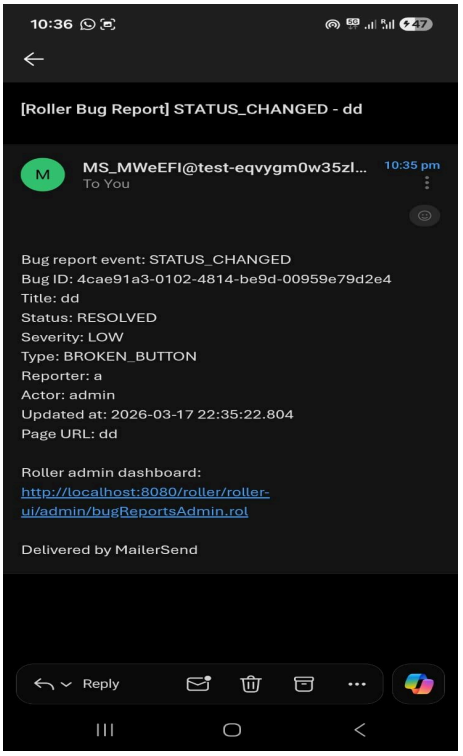
- Bug creation/edit form



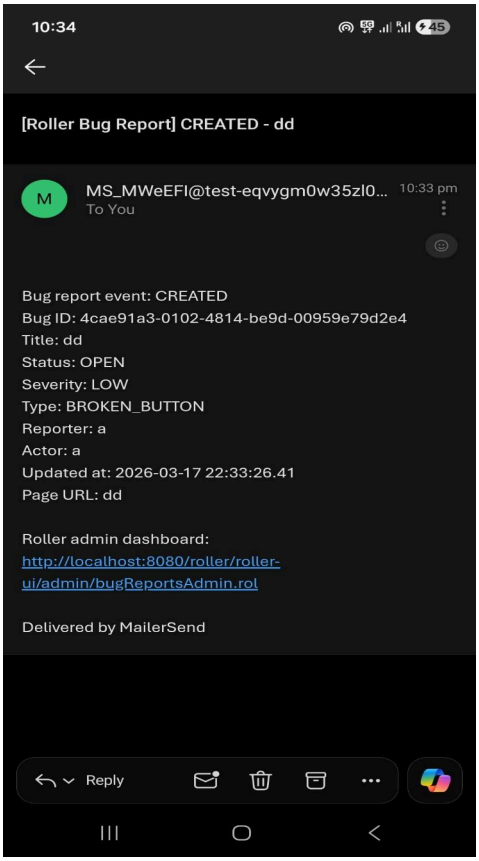
- Admin dashboard



- Status change interface



- Creation mail to admin



3. Assumptions / Constraints

- Users must have `EDIT_DRAFT` permission to access bug reporting
- Admin dashboard requires `ADMIN` permission
- Users can modify only their own reports
- Reporter notifications are sent only on status change or deletion
- Admin notifications are sent for all actions
- Email delivery depends on valid SMTP configuration
- Only one notification channel (`EmailBugNotificationChannel`) is currently implemented
- Status model is limited to OPEN, TRIAGED, and RESOLVED
- User and admin interfaces are separate

---

4. Files Modified / Added

Domain and Persistence

- `BugReport.java`
- `BugReport.orm.xml`
- `createdb.vm`
- `persistence.xml`

Business and Workflow

- `BugReportManager.java`
- `JPABugReportManagerImpl.java`
- `BugReportCommand.java`
- `CreateBugReportCommand.java`
- `UpdateBugReportCommand.java`
- `DeleteBugReportCommand.java`
- `ChangeStatusBugReportCommand.java`
- `BugReportWorkflowService.java`
- `BugReportServiceFactory.java`

Notifications

- BugReportEvent.java
- BugReportEventType.java
- BugReportAudience.java
- BugReportEventPublisher.java
- BugReportNotificationService.java
- BugNotificationChannel.java
- EmailBugNotificationChannel.java
- BugNotificationChannelFactory.java
- BugNotificationMessage.java
- BugNotificationMessageStrategy.java
- AdminBugNotificationMessageStrategy.java
- ReporterStatusNotificationMessageStrategy.java

UI and Views

- BugReportBean.java
- BugReports.java
- BugReportsAdmin.java
- BugReports.jsp
- BugReportEdit.jsp
- BugReportsAdmin.jsp

Framework and Configuration Integration

- struts.xml
- tiles.xml
- editor-menu.xml
- admin-menu.xml
- ApplicationResources.properties
- runtimeConfigDefs.xml
- Weblogger.java
- WebloggerImpl.java
- JPAWebloggerImpl.java
- JPAWebloggerModule.java

Tests and Documentation

- `BugReportTest.java`
- `BugReportCRUDTest.java`
- `BugReportBeanTest.java`
- `BugReportWorkflowServiceNotificationTest.java`
- `EmailBugNotificationSmtpSmokeTest.java`
- `project2_bug-report-task.md`
- `bug-report-module-class-diagram.puml`

---

5. Design Patterns Used

- **Repository Pattern**
  The **Repository Pattern** is implemented through `BugReportManager` and its concrete implementation `JPABugReportManagerImpl`. This pattern abstracts persistence logic from the rest of the application and aligns with Roller's existing manager-based architecture. It ensures that database operations are centralized and isolated from business logic, improving maintainability and consistency across the system.

- **Command Pattern**
  The **Command Pattern** is used extensively in the workflow layer through `BugReportCommand` and its concrete implementations such as `CreateBugReportCommand`, `UpdateBugReportCommand`, `DeleteBugReportCommand`, and `ChangeStatusBugReportCommand`. Each operation on a bug report is encapsulated as a separate command, allowing validation, execution, and testing to be handled independently. This improves modularity and makes the system easier to extend with additional operations in the future.

- **Strategy Pattern**
  The **Strategy Pattern** is applied in the notification subsystem using `BugNotificationMessageStrategy` and its implementations `AdminBugNotificationMessageStrategy` and `ReporterStatusNotificationMessageStrategy`. Since different audiences require different message formats, this pattern enables flexible message generation without modifying the notification pipeline. It improves extensibility and allows new message formats to be introduced easily.
-

- **Observer (Event) Pattern**
  An **Observer (Event-Driven) Pattern** is realized through `BugReportEvent` and `BugReportEventPublisher`. The workflow service generates events when actions

such as create, update, delete, or status change occur, and these events are propagated to the notification system. This decouples the core business logic from notification handling, ensuring that additional reactions to events can be introduced without modifying existing workflows.

- **Factory Pattern**
  The **Factory Pattern** is used in `BugNotificationChannelFactory` and `BugReportServiceFactory` to centralize object creation. This prevents direct instantiation of concrete classes and allows runtime configuration of notification channels. As a result, the system can support additional channels such as Slack or Microsoft Teams with minimal changes.

- **Template Method Pattern**
  Template Method Pattern is leveraged through the existing `UIAction` class provided by Apache Roller. All UI actions inherit common behavior such as security checks, session handling, and request processing, ensuring consistency across user and admin interfaces while reducing code duplication.

---

6. Rationale Behind Pattern Selection

- The Repository pattern aligns with existing Roller architecture and isolates database logic
- The Command pattern encapsulates different operations and improves extensibility
- The Strategy pattern enables flexible notification message formatting
- The Observer pattern decouples workflow from notification delivery
- The Factory pattern centralizes object creation and supports future extensibility
- The Template Method pattern ensures consistency with Roller UI behavior

---

7. User Flow

Normal User Flow

1. User logs into Roller and opens a weblog
2. User navigates to the bug report section
3. User creates a new bug report
4. The system validates and stores the report using `BugReportWorkflowService`
5. A `BugReportEvent` is generated and admins are notified
6. User can edit or delete their report

Admin Flow

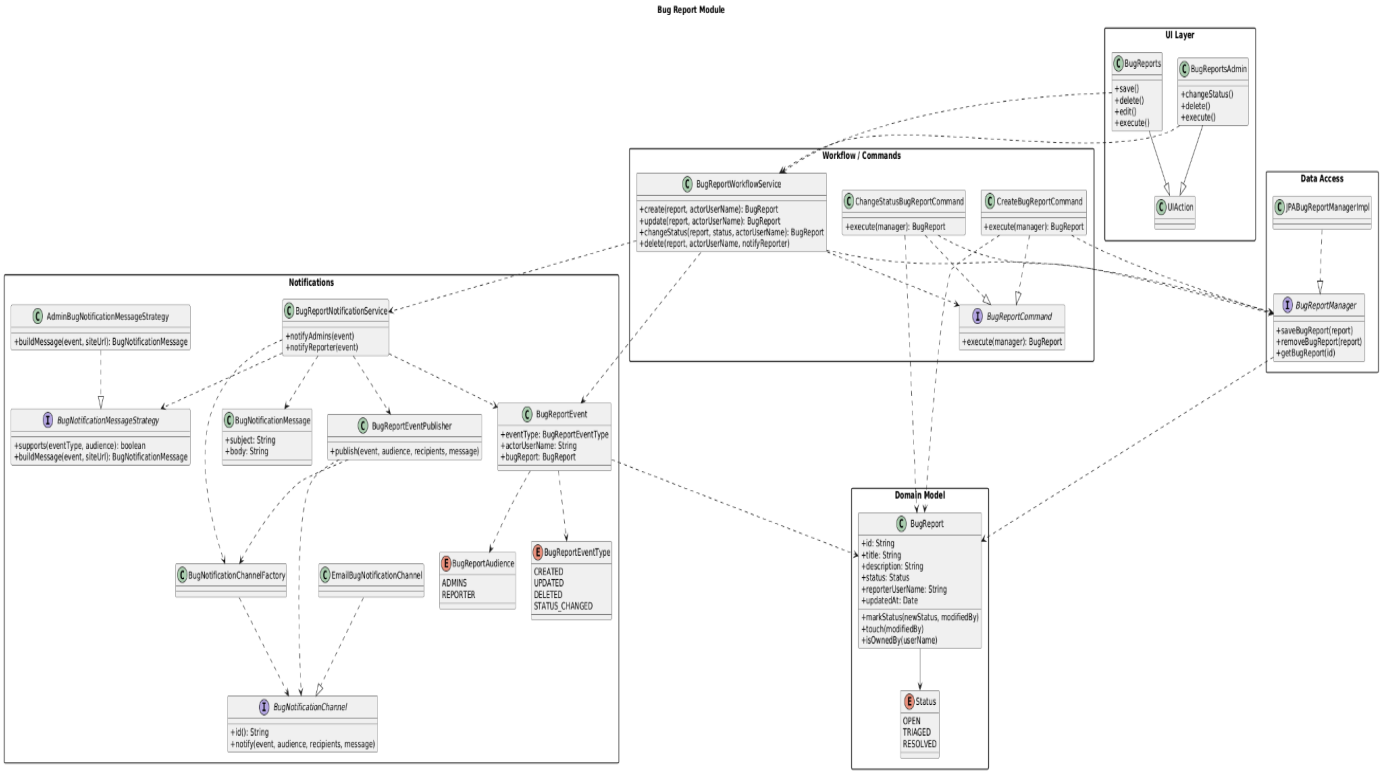
1. Admin logs into Roller and opens the dashboard
2. Admin views or filters reports
3. Admin updates status using `ChangeStatusBugReportCommand`
4. The system persists changes and triggers notifications
5. Admin can delete reports

Notification Flow

1. A workflow action is executed via `BugReportWorkflowService`
2. A `BugReportEvent` is generated
3. `BugReportNotificationService` determines recipients
4. A `BugNotificationMessageStrategy` generates the message
5. `BugReportEventPublisher` sends notifications via `EmailBugNotificationChannel`

---

8. UML Diagram



Task 4: Admin Dashboard (Site Summary)

---

1. Feature Description

The Admin Dashboard provides a centralized Site Summary page for administrators to view key metrics about the entire Roller installation at a glance. The dashboard supports two view modes — Full (9 metrics) and Minimalist (3 metrics) — that can be toggled from the UI.

The implementation uses the Builder design pattern to separate view definition (which metrics to show in each mode) from data-fetching logic (how each metric computes its

value). This means new metrics can be added by implementing a single interface, and new views can be created by changing which metrics the builder includes — neither change affects the other.

Nine metrics are implemented:

Total Registered Users
Total Weblogs
Total Blog Entries
Total Comments
Top Performing Category (by entry count)
Most Commented Category
Most Starred Blog
Top 3 Most Active Users (by weblog ownership)
Most Commented Weblog

---

2. Implementation

Backend

Core Architecture:

The `DashboardMetric` interface defines the contract for all metrics:

```
public interface DashboardMetric {
   String getName();    // e.g., "totalUsers"
   String getLabel();   // e.g., "Total Registered Users"
   MetricResult compute();
}
```

The `MetricResult` class is an immutable value object containing:

name (String) — metric identifier

label (String) — human-readable label

value (String) — the main metric value (e.g., "42")

details (List) — optional additional details (e.g., "150 entries")

The `DashboardReport` class is an immutable container holding:

viewName (String) — "Full" or "Minimalist"

results (List) — computed metric results (unmodifiable)

`getMetricCount()` — number of metrics in the report

The `DashboardReportBuilder` separates view definition from data fetching:

Fluent API: `setViewName(name).addMetric(metric).build()`

Each metric is computed independently; one failure produces an "Error" result without preventing others

Two predefined view constructors:

`buildMinimalistReport()` — 3 metrics: TotalUsers, TotalWeblogs, TopCategory

`buildFullReport()` — all 9 metrics

Metric Implementations:

Metric Class / Data Source / Return Value

`TotalUsersMetric` — `UserManager.getUserCount()` — User count

`TotalWeblogsMetric` — `WeblogManager.getWeblogCount()` — Weblog count

`TotalEntriesMetric` — `WeblogEntryManager.getEntryCount()` — Entry count

`TotalCommentsMetric` — `WeblogEntryManager.getCommentCount()` — Comment count

`TopCategoryMetric` — Iterates weblogs → categories → counts entries per category — Category name + entry count

`MostCommentedCategoryMetric` — Iterates weblogs → categories → entries → sums comment counts — Category name + comment count

`MostStarredBlogMetric` — `StarManager.getTrendingWeblogs(1)` — Weblog name + star count

`TopActiveUsersMetric` — Ranks users by number of owned weblogs — Top 3 user names

`MostCommentedWeblogMetric` — `WeblogManager.getMostCommentedWeblogs()` — Weblog handle + comment count

Struts2 Action:

The `SiteSummary` action class:

Requires ADMIN global permission

Accepts a view parameter (defaults to "full")

Calls `buildFullReport()` or `buildMinimalistReport()` based on the view parameter

Returns the `DashboardReport` to the JSP for rendering

Frontend

SiteSummary.jsp: The JSP uses Roller's standard admin table styling (rollertable table-striped) matching the look of existing admin pages like CacheInfo.jsp and PingTargets.jsp.

The page displays:

A header showing the current view name and metric count

Toggle links between Full and Minimalist views (bold text for active view, link for inactive)

A table with three columns: Metric, Value (bold), and Details

Each metric result is rendered as a table row

[Screenshots to be added: Full view with 9 metrics, Minimalist view with 3 metrics]

---

3. Assumptions / Constraints

Only administrators can access the Site Summary page (enforced by requiredGlobalPermissionActions)

The dashboard does not require a specific weblog context (`isWeblogRequired()` returns false)

Metrics are computed on every page load — no caching layer is implemented

The `TopCategoryMetric` and `MostCommentedCategoryMetric` iterate over all weblogs and categories, which could be slow on very large installations

Metric computation failures are isolated — one failing metric shows "Error" without blocking others

The Minimalist view includes TotalUsers, TotalWeblogs, and TopCategory (3 metrics)

---

4. Files Created and Modified

New Files Created:

Core Architecture:

```
DashboardMetric.java
MetricResult.java
DashboardReport.java
DashboardReportBuilder.java
```

Metric Implementations:

```
TotalUsersMetric.java
TotalWeblogsMetric.java
TotalEntriesMetric.java
TotalCommentsMetric.java
TopCategoryMetric.java
MostCommentedCategoryMetric.java
MostStarredBlogMetric.java
TopActiveUsersMetric.java
MostCommentedWeblogMetric.java
```

UI:

```
SiteSummary.java
SiteSummary.jsp
```

Tests:

```
DashboardReportBuilderTest.java
DashboardReportTest.java
MetricResultTest.java
SiteSummaryActionTest.java
```

Files Modified:

`struts.xml` — added siteSummary action mapping

`tiles.xml` — added .SiteSummary tile definition

`ApplicationResources.properties` — added i18n messages

---

5. Design Patterns Used

Builder Pattern: The `DashboardReportBuilder` implements the Builder pattern with a clear separation between:

View definition — which metrics are included in each view mode (handled by `buildFullReport()` and `buildMinimalistReport()`)

Data-fetching logic — how each metric computes its value (handled by individual `DashboardMetric` implementations)

The builder provides a fluent API (`setViewName().addMetric().addMetric().build()`) that makes view construction readable and composable. The `build()` method computes all added metrics and returns an immutable `DashboardReport`.

Key Builder pattern benefits demonstrated:

Adding a new metric only requires implementing `DashboardMetric` and adding one `addMetric()` call

Adding a new view only requires a new method combining metrics

The builder handles error isolation

---

6. Rationale Behind Pattern Selection

The Builder pattern was chosen because the admin dashboard requires constructing complex objects (reports with varying metric compositions) where:

Construction varies by context — the Full view includes 9 metrics while the Minimalist view includes 3, but they share the same construction process.
Component independence — each metric is self-contained and computes its own data; the builder coordinates without coupling them.
Extensibility — new metrics or views can be added without modifying existing code, supporting the Open/Closed Principle.

Error isolation — the builder wraps each metric computation in a try-catch, so one failing metric doesn't prevent the report from being built.

Alternative patterns considered:

Abstract Factory would work but adds unnecessary interface complexity when there's only one builder
Strategy doesn't fit because the metrics aren't interchangeable alternatives — they're additive components of a report

---

7. User Flow

Viewing the Admin Dashboard:

Admin user logs in and navigates to the admin menu

Admin clicks "Site Summary"

`SiteSummary.execute()` reads the view parameter (default: "full")

Builder assembles the report (`buildFullReport()` or `buildMinimalistReport()`)

Each metric queries its respective manager

The report is rendered in `SiteSummary.jsp`

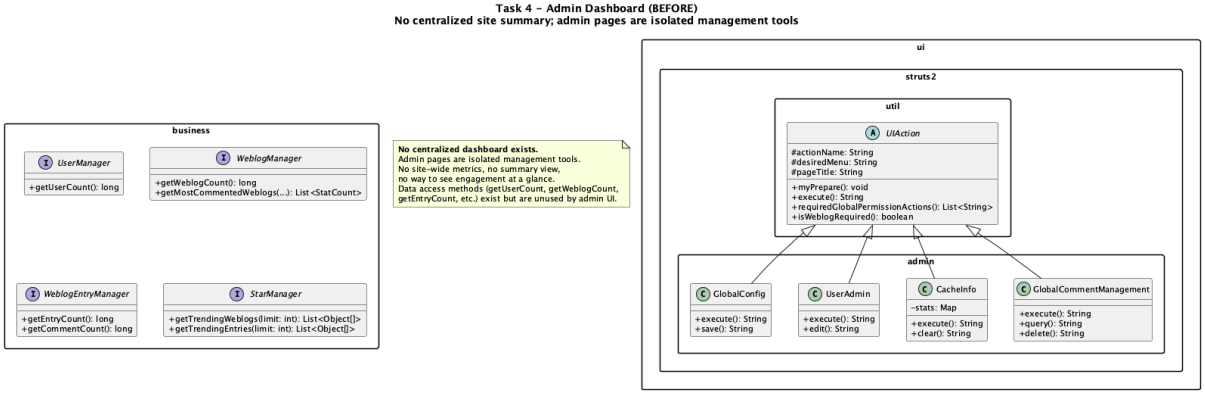
Admin can toggle between views

---

8. Before and After UML Class Diagram

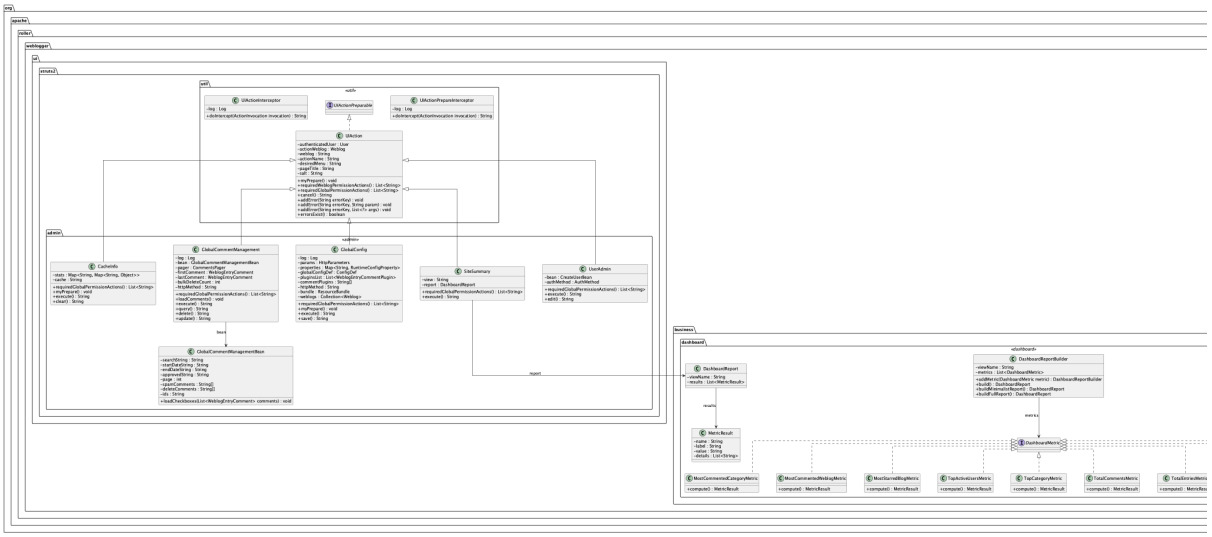
Before (no centralized dashboard):

The existing admin pages are isolated management tools (GlobalConfig, UserAdmin, CacheInfo, GlobalCommentManagement). Data access methods like getUserCount() and getWeblogCount() exist but are not exposed through any dashboard view.



After (Builder-based dashboard with 9 metrics):

The new dashboard package adds the `DashboardMetric` interface, `MetricResult` value object, `DashboardReport` container, `DashboardReportBuilder`, and 9 metric implementations. The `SiteSummary` action connects the dashboard to the admin UI.



Task 5: Web Translation and Caching

1. Feature Description

Task 5 introduces a translation subsystem for public Roller weblog pages and combines it with section-level caching to avoid repeated calls to external translation providers.

The feature allows users to open a public weblog page, select a source language, target language, and translation provider from a floating widget, and translate the visible text content without altering the page structure.

The implementation supports translation across multiple public views including weblog pages, permalinks, search pages, tags, archives, and frontpage views.

Key capabilities include:

- Translation of rendered weblog text on public pages
- Runtime switching between `MyMemoryTranslationProvider` and `SarvamTranslationProvider`
- Support for multiple languages including Marathi
- Restore-to-original functionality
- Automatic reuse of previously selected translation settings
- Section-level caching to avoid redundant translations

---

2. Implementation

Backend

The backend is built around a translation pipeline exposed through `TranslationServlet`.

The `TranslationServlet` receives JSON requests from the frontend, validates language inputs, selects the appropriate provider using `TranslationProviderFactory`, and delegates translation to `TranslationCacheService`.

Core Components

Translation Entry Point
The `TranslationServlet` handles request parsing, validation, provider resolution, and response generation.

Caching Layer
The `TranslationCacheService` performs section-level caching. It computes a SHA-256 hash over normalized text content (whitespace-trimmed and normalized) to detect meaningful changes.

Cache keys include:

- provider
- source language
- target language
- content hash

This ensures cached translations are reused only when the content is unchanged.

Language Normalization
The `TranslationLanguageSupport` class standardizes language codes (e.g., `en-US`, `mr_IN`) and maps them to provider-specific formats such as `mr-IN`.

Provider Layer
The `TranslationProvider` interface defines a common contract for translation providers.

Concrete implementations include:

- `MyMemoryTranslationProvider`
- `SarvamTranslationProvider`

The `SarvamTranslationProvider` reads API keys from Roller configuration, while both providers support batch translation while preserving input order.

Backend Functionality

- `TranslationServlet` supports section-based translation requests
- `TranslationProviderFactory` selects providers dynamically
- `TranslationCacheService` reuses cached sections and retranslates only changed content
- `TranslationLanguageSupport` ensures consistent language handling
- Providers perform batch translation and preserve response order

---

Frontend

The frontend is implemented as a shared JavaScript widget using:

- `translation.js`
- `translation.css`

The widget is injected on page load and performs the following steps:

- Detects source language from page attributes
- Extracts visible text nodes and groups them into sections
- Sends sections to `/roller-services/translate`

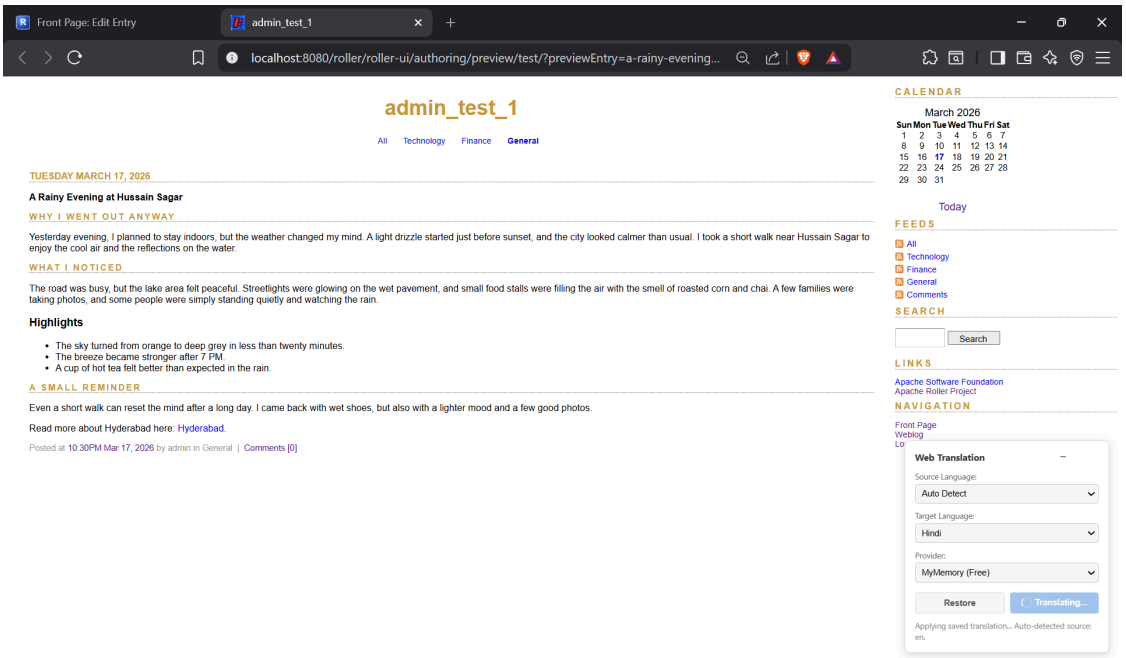
- Replaces translated text in the DOM without altering layout

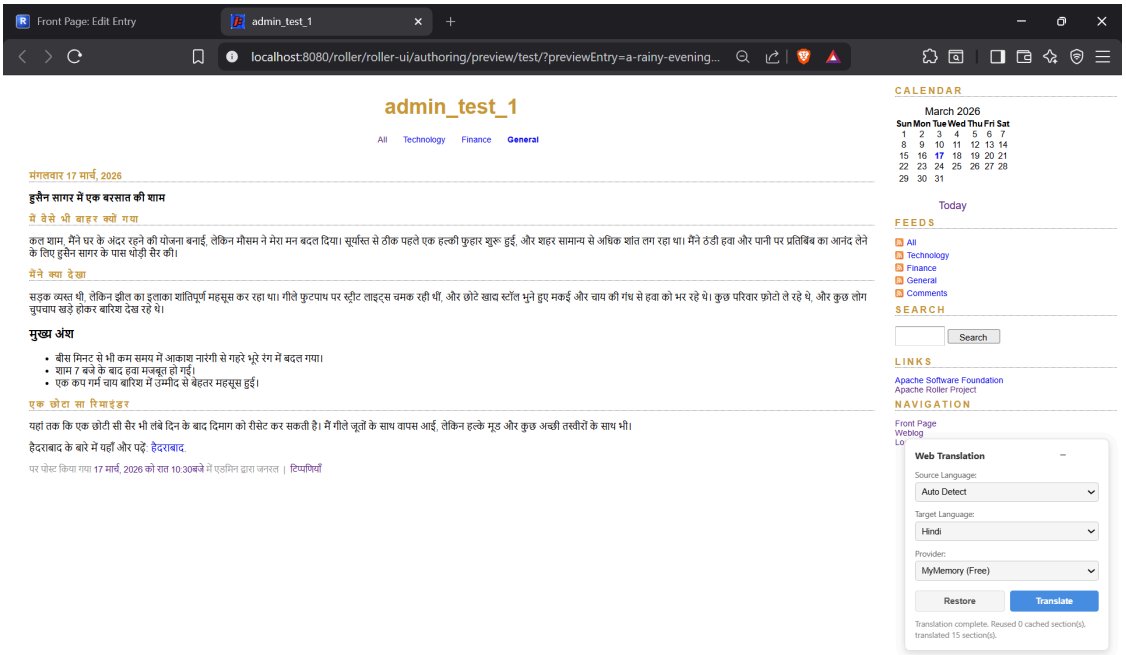
Additional features include:

- Local storage of translation preferences
- Restore-to-original functionality
- Automatic reapplication of translation on revisit

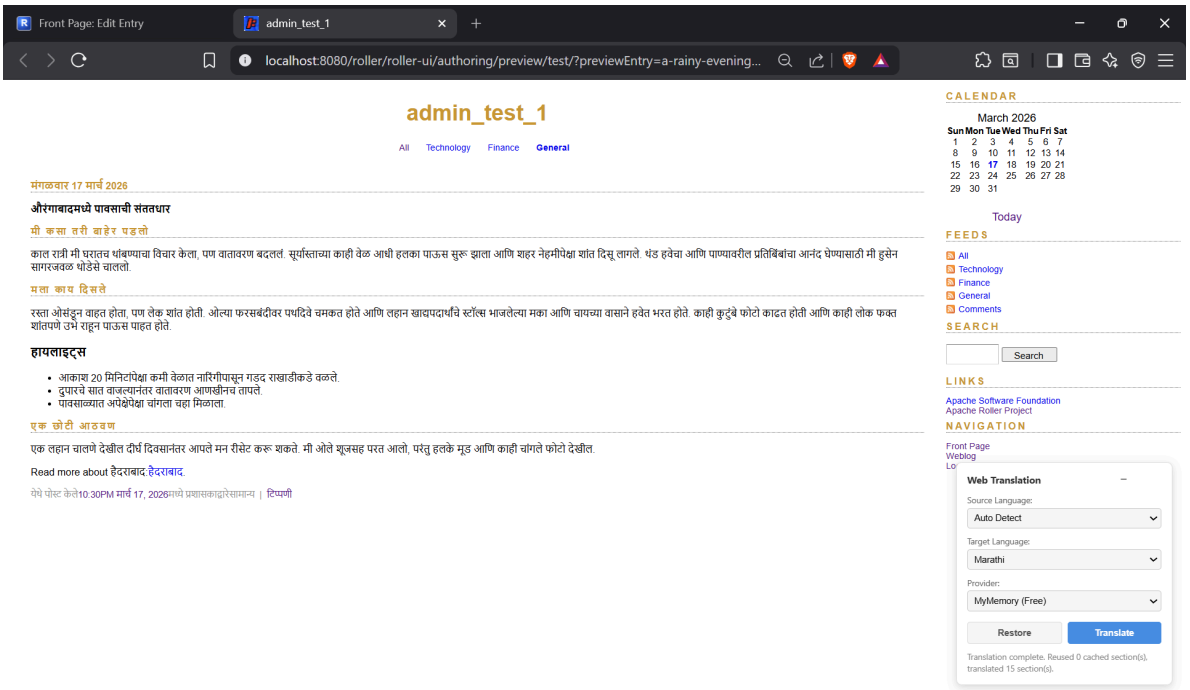
Screenshots

1. Webpage with translation widget

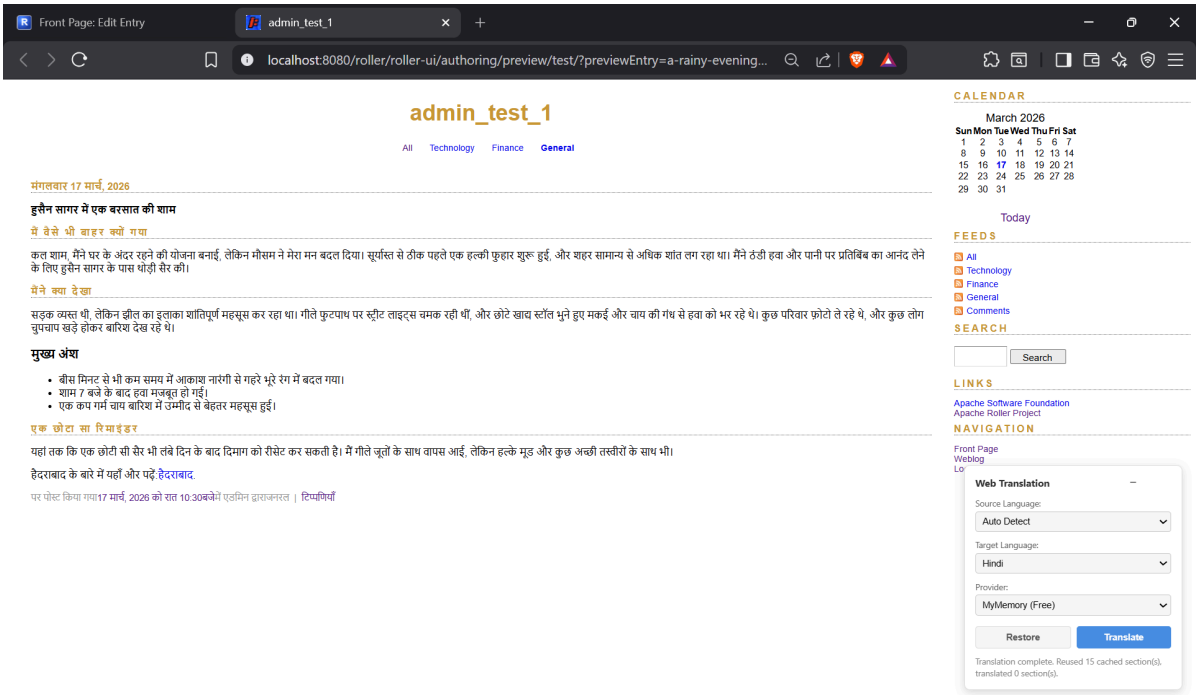




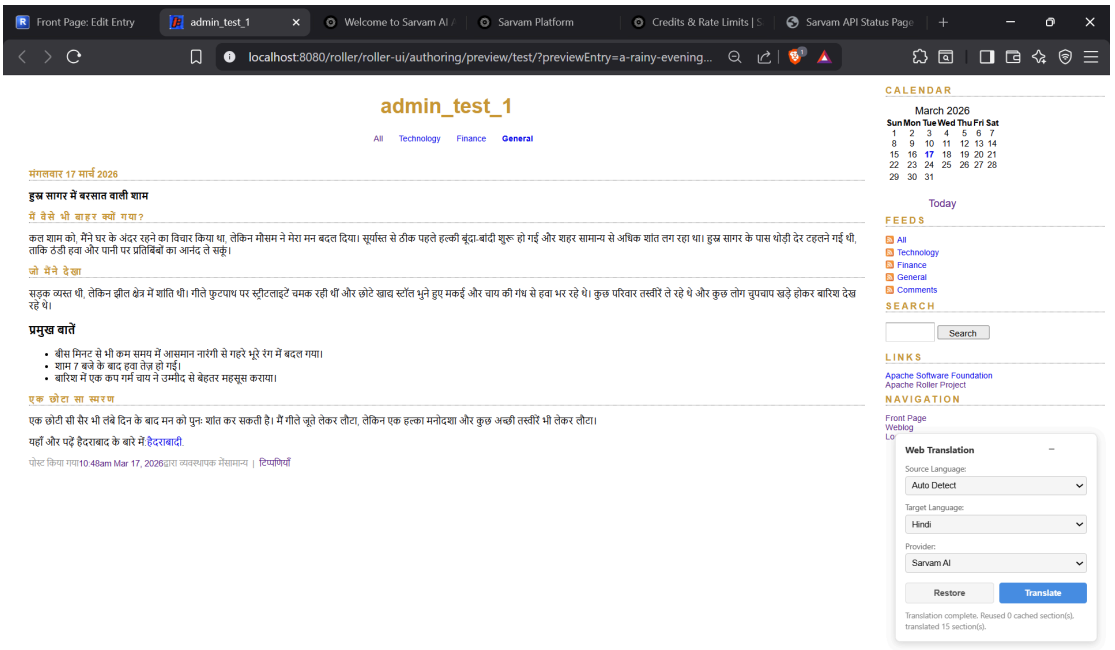
2. Translated page (Marathi)



3. Cached translation reuse indicator



4. Provider switch (`MyMemoryTranslationProvider` vs `SarvamTranslationProvider`)



---

3. Assumptions / Constraints

- Translation is available only on theme views that include the widget
- Supported languages: `en`, `hi`, `bn`, `ta`, `te`, `kn`, `mr`
- Source language detection uses page `lang` attribute first
- Only visible text nodes are translated (no images, links, attributes)
- Cache is in-memory within `TranslationCacheService`
- Cache is not persistent across server restarts
- Cache reuse depends on normalized content and translation parameters
- `SarvamTranslationProvider` requires a valid API key
- Frontend integration is limited to selected theme views

---

4. Files Modified / Added

Backend

- `TranslationServlet.java`
- `TranslationCacheService.java`
- `TranslationLanguageSupport.java`
- `TranslationProvider.java`
- `TranslationProviderFactory.java`
- `MyMemoryTranslationProvider.java`
- `SarvamTranslationProvider.java`
- `TranslationSectionRequest.java`
- `TranslationSectionResponse.java`
- `web.xml`
- `roller.properties`

Frontend

- `translation.js`
- `translation.css`

Theme Integration

- `weblog.vm`
- `permalink.vm`
- `searchresults.vm`

- `archives.vm`
- `tags_index.vm`
- `_header.vm`, `_footer.vm`

Tests and Documentation

- `TranslationCacheServiceTest.java`
- `TranslationLanguageSupportTest.java`
- `task5-translation-system.md`
- `task5-translation-system-class-diagram.puml`

---

5. Design Patterns Used

- **Strategy Pattern**
  The **Strategy Pattern** is used through the `TranslationProvider` interface and its implementations, `MyMemoryTranslationProvider` and `SarvamTranslationProvider`. This allows the system to support multiple translation providers with different APIs and configurations while exposing a uniform interface to the rest of the application. As a result, the translation logic remains independent of provider-specific details, improving modularity and enabling easy extension to additional providers in the future.

- **Factory Pattern**
  The **Factory Pattern** is implemented via `TranslationProviderFactory`, which centralizes the creation of translation provider instances. This ensures that the servlet layer depends only on abstractions rather than concrete implementations, thereby reducing coupling and improving maintainability. It also simplifies the integration of new providers by localizing object creation logic.

- **Cache Pattern (Decorator-like behavior)**
  The Cache Pattern is realized through `TranslationCacheService`, which stores translations at a section level to avoid redundant API calls. By computing a hash of normalized content, the system ensures that translations are reused only when the content remains unchanged. This significantly improves performance, reduces external API usage, and enhances responsiveness for repeated page visits, while

introducing additional complexity in cache management and invalidation.

- **Adapter-like Pattern**
  A **Template Method–style workflow** is followed in `TranslationServlet`, which defines a fixed sequence of steps for handling translation requests, including request parsing, language normalization, provider selection, cache lookup, and response generation. While not implemented through classical inheritance, this structured pipeline ensures consistency and improves readability and maintainability of the request-handling process.

- **Template Method Style Flow**
  Adapter-like pattern is employed in `TranslationLanguageSupport` to normalize and map language codes across different system components. Since the frontend, internal system, and external providers use varying language formats, this layer ensures compatibility and centralizes language handling logic, improving maintainability and reducing errors.

---

6. Rationale Behind Pattern Selection

- Strategy allows switching between translation providers without modifying core logic
- Factory centralizes provider selection and simplifies extensibility
- Cache reduces redundant API calls and improves performance
- Normalization layer ensures compatibility between frontend and provider formats
- Template-style flow keeps request processing structured and maintainable

---

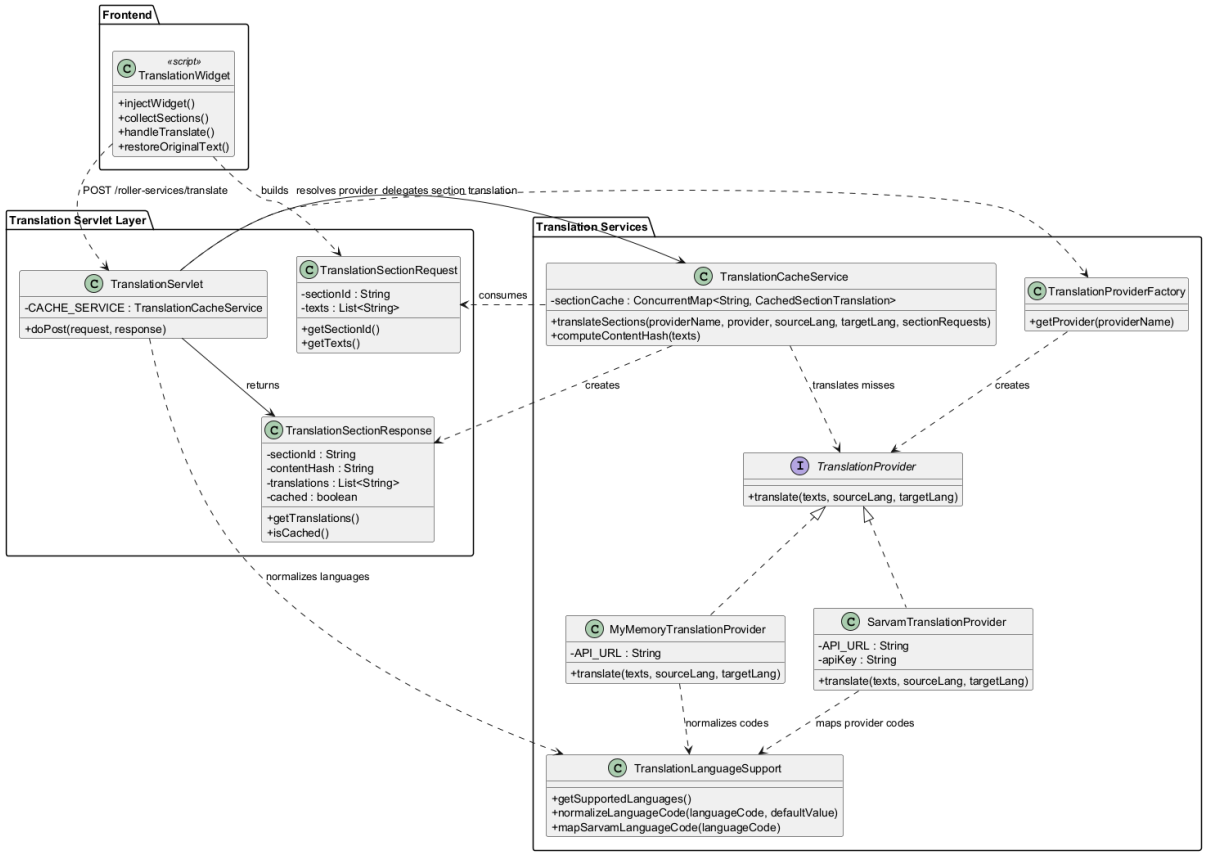
7. User Flow

1. User opens a public weblog page with translation widget
2. `translation.js` initializes and restores saved preferences
3. User selects target language and provider
4. Text content is extracted and grouped into sections
5. Sections are sent to `/roller-services/translate`
6. `TranslationServlet` processes the request and invokes `TranslationCacheService`
7. Cached sections are reused, and only new content is translated
8. Translated content is returned to frontend
9. DOM text is replaced without altering layout
10. Settings are saved for future visits

11. If content changes, only modified sections are retranslated

---

8. UML Diagram



Task 6: Community Pulse Dashboard

Task 6A — Discussion Overview Indicators

1. Feature Description

The Discussion Overview feature provides five lightweight indicators that summarize the discussion around a blog entry's comments. All indicators use classical, computationally inexpensive methods (keyword frequency, statistics, regex patterns) — no LLM involvement.

The five indicators are:

1 .Activity Level — classifies discussion intensity (Silent, Cold, Warm, Hot, On Fire)
2. Response Types — categorizes comments as questions, positive, debate, or general
3. Recurring Keywords — extracts the top 5 most frequently occurring meaningful words
4. Top Contributors — identifies the top 3 most active commenters
5. Unique Commenters — measures engagement diversity with a diversity ratio

---

2. Implementation

Backend

Core Interface:

```
public interface DiscussionIndicator {
   String getName();   // e.g., "activityLevel"
   String getLabel();  // e.g., "Discussion Activity Level"
   Map<String, Object> compute(CommentData data);
}
```

The `CommentData` class is an immutable snapshot containing:

entryId and entryTitle — entry metadata
entryPubTime — publication timestamp
comments — list of `WeblogEntryComment` objects (approved only)

The `DiscussionOverview` aggregator registers all 5 indicators and calls `computeAll(data)` to produce a map of indicator name → result map.

Indicator Implementations:

Indicator	Method	Output Keys
`ActivityLevelIndicator`	Comment count + comments-per-day rate	level, totalComments, commentsPerDay
`ResponseTypeIndicator`	Regex pattern matching for questions/positive/debate/general	questions, positive, debate, general (counts + percentages)
`RecurringKeywordsIndicator`	Word frequency analysis with stop-word filtering	keywords (list of top 5 with counts), totalUniqueWords
`TopContributorsIndicator`	Commenter name frequency	contributors (list of top 3 with counts), totalContributors
`UniqueCommenterIndicator`	Set-based unique name counting	uniqueCount, totalComments, diversityRatio, diversityLabel

---

Activity Level Thresholds (composite — based on both comment count and comments-per-day rate):

Silent: 0 comments
Cold: ≤3 comments and rate < 0.5/day
Warm: ≤10 comments and rate < 2.0/day
Hot: ≤30 comments and rate < 5.0/day
On Fire: >30 comments or rate ≥ 5.0/day

Frontend

The Community Pulse JSP page displays all 5 indicators in a panel layout when an entry is analyzed. Each indicator appears as a card with its label and computed values.

---

3. Assumptions / Constraints

- Only approved comments are analyzed (spam and pending comments are excluded)
- Anonymous commenters are tracked by the name they provide (may have duplicates if different people use the same name)
- Keyword extraction uses English stop words only — non-English comments may produce less meaningful results
- Activity level thresholds are hardcoded (not configurable)
Response type classification uses simple regex patterns and may not capture nuanced language

---

Task 6B — Conversation Breakdown

---

1. Feature Description

The Conversation Breakdown feature extracts major themes from blog entry comments, identifies representative comments for each theme, and generates a short recap of the overall discussion. Two distinct breakdown methods are implemented:
TF-IDF Breakdown (lightweight) — uses term frequency-inverse document frequency analysis for clustering
Hybrid LLM Breakdown (heavy) — combines local TF-IDF clustering with LLM-enhanced theme labeling via Gemini API
The system automatically selects the appropriate method based on comment count and LLM availability, and users can override the selection from the dashboard.

---

2. Implementation

Backend

Strategy Interface:

```
public interface BreakdownStrategy {

   String getName();
```

```
    String getDescription();

    ConversationBreakdown analyze(CommentData data);

}
```

`ConversationBreakdown` contains:

themes — list of `ConversationTheme` objects
recap — 2-3 sentence summary of the discussion
methodUsed — strategy name used for analysis

`ConversationTheme` contains:

label — human-readable theme name
keywords — list of representative keywords
representativeComments — sample comments per theme (`TfIdfBreakdownStrategy` selects up to 2 per theme via REPS_PER_THEME)
commentCount — number of comments in this theme

Method 1: `TfIdfBreakdownStrategy` ("tfidf"):

Tokenizes comments with stop-word filtering
Computes IDF (inverse document frequency) across all comments
Calculates TF-IDF vectors per comment
Clusters comments by highest keyword scores
Extracts top 2 representative comments per theme (by TF-IDF score)
Generates recap from cluster summaries
Maximum 5 themes
No external API calls — fully local computation

Method 2: `HybridLlmBreakdownStrategy` ("hybrid-llm"):

Runs TF-IDF locally first (free computation)
Builds a compact prompt with cluster keywords + 2 sample comments per cluster
Calls Gemini 2.0 Flash API for polished theme labels and a discussion recap
Falls back to TF-IDF labels if LLM call fails
Temperature: 0.3 (deterministic), max tokens: 500
Cost optimization: sends only keywords + samples (minimal tokens), not full comments

---

Strategy Selection (`BreakdownStrategySelector`): The selector acts as a Factory + Strategy pattern implementation:

0-5 comments → TF-IDF only (too few for meaningful clustering)
6-30 comments → TF-IDF by default; Hybrid if LLM is configured
31+ comments → Hybrid preferred (LLM adds value for large discussions); falls back to TF-IDF

Users can override via strategy parameter in the UI

Caching (`CommunityPulseAnalyzer`): The `CommunityPulseAnalyzer` facade coordinates both 6A and 6B:

Static ExpiringLRU cache (survives across action invocations)
Cache key: entryId:strategyName
TTL: 6 hours, max size: 50 entries
Invalidation: cache is stale if 20+ new comments arrived since last analysis
Manual refresh flag bypasses cache entirely

Frontend

`CommunityPulse.jsp`: The JSP provides:

Entry selector dropdown (recent entries)
6A Overview: 5-panel layout showing all indicators
6B Breakdown: Theme cards with keywords, representative comments, and recap
Refresh button (bypass cache)
Strategy switcher dropdown (when 2+ methods are available)

CommunityPulse Struts Action:

`execute()` — lists recent entries for selection
`analyze()` — runs full analysis (6A + 6B) for a selected entry

Inputs: entryId, strategy (optional), refresh (boolean)
Outputs: pulseResult, recentEntries, cached result flag

[Screenshots to be added: Community Pulse page with entry selector, 6A indicators panel, 6B breakdown with themes and recap, strategy switcher]

---

3. Assumptions / Constraints

Analysis is per-entry (not per-blog or per-site)
Only approved comments are analyzed
TF-IDF clustering uses English stop words; non-English discussions may produce less

meaningful themes
- The Hybrid LLM strategy requires a valid Gemini API key configured in roller.properties
- If the LLM API call fails, the Hybrid strategy falls back to TF-IDF labels (no user-facing error)
- Cache invalidation threshold of 20 new comments is hardcoded
- Maximum 5 themes are extracted per analysis
- Representative comments are selected by TF-IDF score (most relevant to the theme, not necessarily most recent)

---

4. Files Created and Modified

New Files Created:

Core Interfaces and Models:

```
DiscussionIndicator.java
BreakdownStrategy.java
CommentData.java
PulseResult.java
ConversationBreakdown.java
ConversationTheme.java
```

6A Indicators:

```
DiscussionOverview.java
ActivityLevelIndicator.java
ResponseTypeIndicator.java
RecurringKeywordsIndicator.java
TopContributorsIndicator.java
UniqueCommenterIndicator.java
```

6B Breakdown Strategies:

```
TfIdfBreakdownStrategy.java
HybridLlmBreakdownStrategy.java
BreakdownStrategySelector.java
```

Orchestration:

`CommunityPulseAnalyzer.java`
`CachedPulseEntry.java`

UI:

`CommunityPulse.java`
`CommunityPulse.jsp`

Tests:

`DiscussionOverviewTest.java`
`ActivityLevelIndicatorTest.java`
`ResponseTypeIndicatorTest.java`
`RecurringKeywordsIndicatorTest.java`
`TopContributorsIndicatorTest.java`
`UniqueCommenterIndicatorTest.java`
`TfIdfBreakdownStrategyTest.java`
`BreakdownStrategySelectorTest.java`
`TestHelper.java`

Files Modified:

`struts.xml` — added communityPulse action mapping (allowed-methods: execute, analyze)

`tiles.xml` — added .CommunityPulse tile definition

`ApplicationResources.properties` — added i18n messages for Community Pulse UI

---

5. Design Patterns Used

Strategy Pattern: The `BreakdownStrategy` interface and its two implementations (`TfIdfBreakdownStrategy`, `HybridLlmBreakdownStrategy`) implement the Strategy pattern. This allows the system to switch between analysis methods at runtime based on:
Comment count (automatic selection)
LLM availability (configuration-driven)
User preference (manual override from UI)
Both strategies implement the same `analyze(CommentData)` method, making them fully interchangeable.

Factory Pattern: The `BreakdownStrategySelector` acts as a Factory that dynamically selects the appropriate strategy based on runtime conditions (comment count, LLM configuration). It encapsulates the selection logic and provides:
`selectStrategy(commentCount)` — automatic selection
`getStrategyByName(name)` — explicit selection
`getAvailableStrategies()` — lists all strategies for UI dropdown

Facade Pattern: The `CommunityPulseAnalyzer` provides a single entry point that coordinates multiple subsystems:
6A: Discussion Overview (5 indicators)
6B: Conversation Breakdown (strategy selection + analysis)
Caching layer (ExpiringLRU cache with smart invalidation)
Comment data fetching (single DB query shared by all components)

The facade hides the complexity of these subsystems behind a simple `analyze(entryId)` method.

---

6. Rationale Behind Pattern Selection

Strategy Pattern was chosen for the breakdown methods because the two approaches (TF-IDF and Hybrid LLM) solve the same problem (extracting themes from comments) using fundamentally different techniques with different resource requirements. The Strategy pattern allows:
Runtime selection based on context (comment count, LLM availability)
Easy addition of new strategies without modifying existing code
User override from the dashboard UI
Graceful fallback (Hybrid → TF-IDF) when LLM is unavailable

Factory Pattern complements Strategy by encapsulating the selection logic. The thresholds and decision tree are centralized in `BreakdownStrategySelector`, keeping the action layer thin and focused on request handling.

Facade Pattern was chosen for `CommunityPulseAnalyzer` because the Community Pulse feature involves coordinating multiple independent subsystems (indicators, breakdown, caching, data fetching). Without the facade, the Struts action would need to manually coordinate all of these, leading to a complex and fragile controller.

---

7. User Flow

Analyzing a Blog Entry's Discussion:

Blog author navigates to the Community Pulse page from the editor menu
The page displays a dropdown of recent entries with comments
Author selects an entry and clicks "Analyze"

`CommunityPulse.analyze()` calls `CommunityPulseAnalyzer.analyze(entryId)`
The analyzer checks the cache:
If cached and < 20 new comments: returns cached result instantly
If stale or cache miss: proceeds to full analysis

For a full analysis:
a. Fetches all approved comments for the entry (single DB query)
b. Creates `CommentData` snapshot
c. Runs all 5 Discussion Indicators (6A) on the comment data
d. Selects the appropriate breakdown strategy based on comment count and LLM config
e. Runs the selected strategy to extract themes, representative comments, and recap
f. Caches the result for future requests

The result is displayed on the JSP:
6A: Five indicator panels showing activity level, response types, keywords, contributors, diversity
6B: Theme cards with keywords and sample comments, plus an overall recap

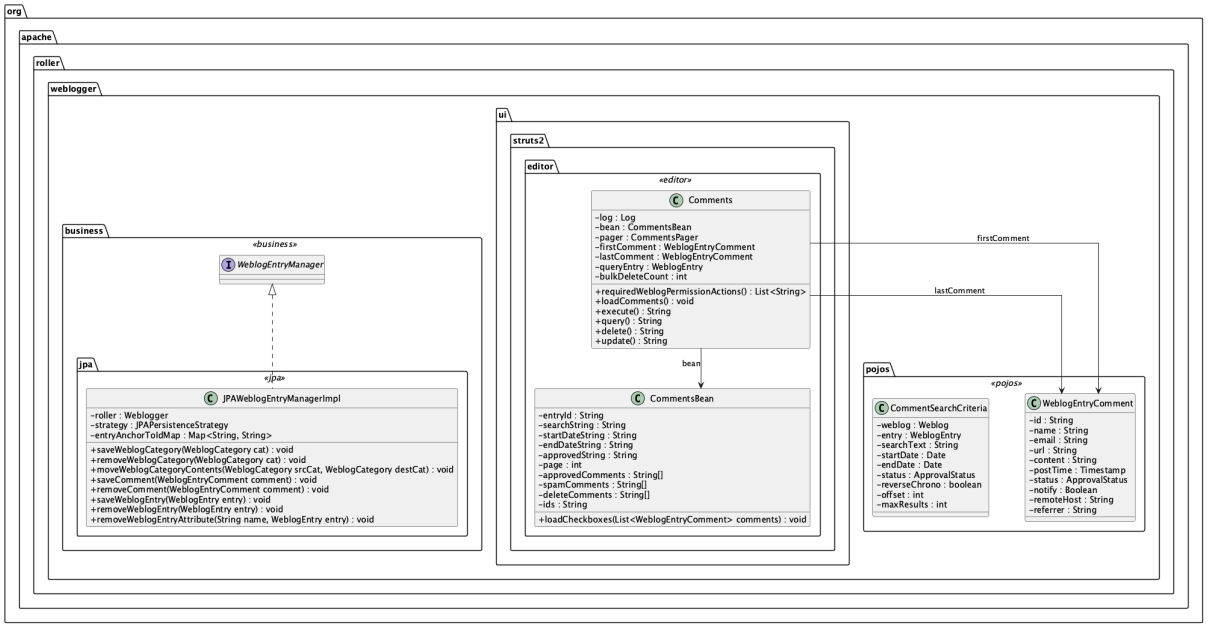
Author can click "Refresh" to bypass cache, or switch strategies via the dropdown

---

8. Before and After UML Class Diagram

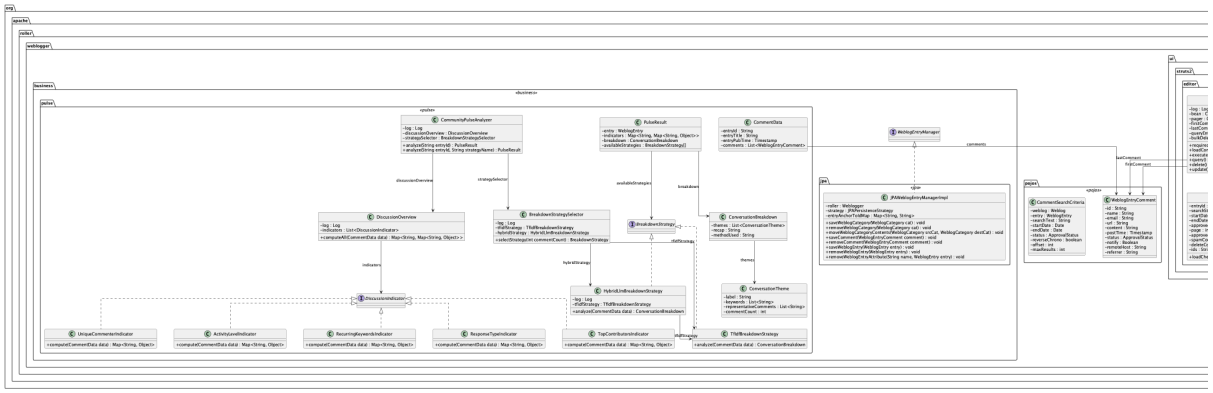
Before (only basic comment management exists):

The existing system has `WeblogEntryManager` for managing comments and `WeblogEntryComment` as the data model. The editor Comments action provides basic listing, deletion, and approval. There is no comment analysis, theme extraction, or discussion summarization capability.



After (full Community Pulse dashboard):

The new pulse package adds the `DiscussionIndicator` interface (5 implementations), `BreakdownStrategy` interface (2 implementations), `BreakdownStrategySelector` factory, `DiscussionOverview` aggregator, `CommunityPulseAnalyzer` facade, and supporting data models. The `CommunityPulse` Struts action connects the analysis to the editor UI.



Bonus 1: Weblog Q&A Chatbot Project Documentation

1. Feature Description

This bonus feature adds a grounded question-answering chatbot to public Roller weblog pages. Instead of manually browsing multiple entries, a reader can ask natural-language questions such as "What has this blog said about data privacy?" and receive an answer based only on the current weblog's published entries.

The implemented scope focuses on public weblog-facing pages. The feature is **grounded by design**: answers are built from published weblog content, and the UI shows supporting source citations so the reader can trace the response back to the original entries.

Main user-visible capabilities:

- Asking free-form questions about the current weblog's published archive.
- Switching between two answering strategies: `RAG` and `Long Context`.
- Using `Auto Pick` to let Gemini choose the better strategy for the question.
- Returning grounded answers with cited source entries.
- Keeping the chatbot embedded directly on public weblog pages.

---

2. Implementation

Backend

The backend is centered around a Q&A pipeline exposed by `WeblogQAServlet`. The servlet delegates the actual answer generation to `WeblogQaService`.

The two implemented answering strategies are:

1. **RetrievalAugmentedWeblogQaStrategy**: Retrieves and ranks the most relevant passages first, then sends that focused context to Gemini.
2. **LongContextWeblogQaStrategy**: Scans a much larger portion of the weblog archive (up to a context budget) for a broader synthesis.

`Auto Pick` is implemented via `GeminiQaAutoStrategySelector`. Gemini first examines the user prompt to choose the best strategy and provide a reason. If Gemini is unavailable, the selector falls back to a heuristic rule set.

Frontend

The frontend is a shared JavaScript widget (`weblog-qa.js`) and CSS (`weblog-qa.css`). The widget is injected on page load, detects the weblog handle from the URL, and renders a floating chat panel.

Report-ready screenshot slots:

1. Public weblog page with the Q&A widget visible.
2. Same question answered using `RAG`.
3. Same question answered using `Long Context`.
4. `Auto Pick` example showing the chosen strategy and the reason.

---

3. Assumptions / Constraints

- Answers are grounded **only** in published entries from the current weblog.
- The feature does not use comments, admin pages, or media metadata.

- Gemini requires a configured API key in `translation-api.properties`.
- Very large archives may be truncated based on the context budget.

---

4. Files Modified / Added

Backend

- `WeblogQAServlet.java`
- `WeblogQaService.java`
- `WeblogQaAnswer.java`
- `WeblogQaSource.java`
- `WeblogQaAnswerStrategy.java`
- `WeblogQaEntryRepository.java`
- `BusinessLayerWeblogQaEntryRepository.java`
- `WeblogQaEntryDocument.java`
- `WeblogQaTextSupport.java`
- `RetrievalAugmentedWeblogQaStrategy.java`
- `LongContextWeblogQaStrategy.java`
- `GeminiQaSynthesisSupport.java`
- `GeminiQaAutoStrategySelector.java`
- `LocalApiPropertiesSupport.java`
- `web.xml`

Frontend & Themes

- `weblog-qa.js` / `weblog-qa.css`
- Modified Velocity templates (`.vm`) for: `basic`, `basicmobile`, `fauxcoly`, `gaurav`, and `frontpage`.

---

5. Design Patterns

Strategy Pattern

Implemented via `WeblogQaAnswerStrategy`. This allows the system to switch between context-construction methods (`RAG` vs. `Long Context`) without changing the servlet contract.

Repository Pattern

Implemented via `WeblogQaEntryRepository`. This decouples answer generation from the underlying Roller business-layer APIs and database access.

Adapter Pattern

Used in `GeminiQaSynthesisSupport` to translate internal data structures into Gemini-compatible REST payloads and vice-versa.

---

6. User Flow

1. **Access:** Reader opens a public weblog page.
2. **Input:** Reader enters a question and selects a strategy.
3. **Processing:** `WeblogQAServlet` receives the request; `WeblogQaService` coordinates data loading.
4. **Synthesis:** The selected strategy builds context and calls the Gemini API.
5. **Output:** The widget renders the answer, source links, and strategy metadata.

---

7. Verification

Targeted Tests: `mvn -pl app "-Dtest=WeblogQaServiceTest,GeminiQaSynthesisSupportTest" test`

Manual Demo Steps:

1. Start Roller with Jetty.
2. Open a weblog with multiple posts.
3. Ask a focused question using `RAG`.
4. Ask a broad summary question using `Auto Pick` and verify the explanation provided by the AI.